Component 2 — Defense and Awareness: Complete Design Document

Oct 9, 2026 · @SVcyber

1. Overview

Component 2 (Defense and Awareness) is the part of the system that decides what to do about a threat and what to tell the user about it. It does not detect scams or hidden images itself — that work belongs to Component 1 (Detection Pipeline) and Component 3 (Steganographic Image Detection). C2's job starts once one of those two components has already found something worth acting on.

From that point, C2 does three things, every time, in one pass: it decides **how serious** the finding is and **what defense action** to take; it generates a **plain-language explanation** that shows the user exactly why this was flagged; and it updates the user's own **security profile**, so patterns in their behaviour over time become visible rather than invisible.

This document uses the official numbering throughout: C1 is Detection, C2 is this component (Defense and Awareness), and C3 is Steganographic Image Detection

This document captures the complete, corrected design for C2 as worked through in detail: how it decides severity and action, how it connects to both C1 and C3 (which turn out to work differently from each other), how its three engines work internally, how its dashboard should be built, and the decisions on where machine learning belongs and where it deliberately does not.

2. Design Philosophy: Why, How, and Logical

Most everyday security software works the same way: it decides something is dangerous, and it either blocks it silently or shows a vague warning ("this message looks suspicious"). The user is rarely told *why*, almost never shown *how* that conclusion was reached, and often left confused about what actually happened to their email. This is the exact gap C2 is built to close, and it is the single idea every part of this document comes back to.

Every time C2 reaches a conclusion, it is built to answer three questions together, not just one:

- **What** — which category this threat falls into (for example, a social-engineering scam, a credential-phishing attempt, or a steganographic image).
- **Why** — the specific evidence and indicators that justify that category. Not a vague feeling of risk, but the actual signals found: a lookalike sender domain, an urgent request to act immediately, a login form embedded in the message, a region of an image whose statistics don't match a normal photograph.
- **How** — the reasoning that connects the evidence to the conclusion, so the logic itself is visible. The user isn't just told "this is dangerous"; they can see what led to that judgement.

This same evidence is used twice, for two different purposes: once to decide **how serious the situation is and what action to take**, and once to decide **what to explain to the user and why it matters to them**. Because both uses draw from the same evidence, the action C2 takes and the explanation C2 gives are always consistent with each other — the user is never shown an explanation that doesn't match what actually happened to their email.

This philosophy is also why C2 never acts on a single number alone. A risk score by itself can be wrong — a false positive, a misread signal. Treating a score as proof and blocking immediately is simple, but it isn't reliable. C2 is built to reason, not just react.

3. The Three Core Engines

C2 is built from three engines that all run from the same incoming evidence, in the same pass, each answering a different question:

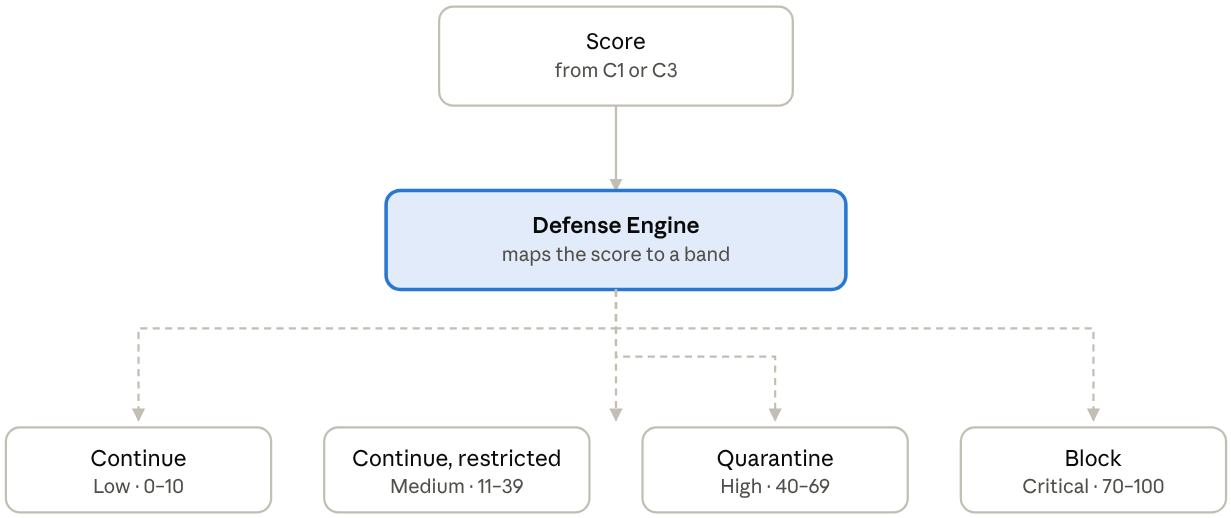
Engine	Question it answers	Output
Defense Engine	What should happen to this email right now?	An action: continue normally, continue with a restriction, quarantine, or block
Awareness Engine	What should the user understand about this?	A plain-language, evidence-based explanation of the scam type and the specific indicators behind it
Vulnerability Profiler	What does this mean for the user over time?	An updated personal security profile showing whether the user is getting safer, staying the same, or needs more support

All three run together, from the same event, but they are independent of each other in one important way: the Defense Engine's action decision is never delayed waiting for the Awareness Engine to finish generating an explanation. If the explanation step is slow, the defensive action still happens immediately. Education and defense are equally important outputs of C2, but defense is time-sensitive in a way explanation is not, so they are deliberately decoupled internally even though they run from the same evidence and reach the user together.

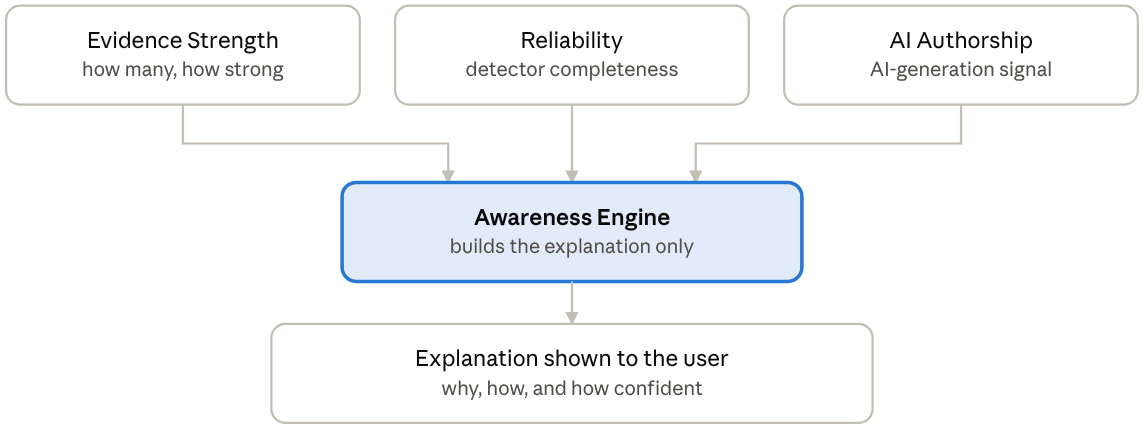
The sections that follow go through each of these in depth: Section 4 explains how the Defense Engine's severity decision actually works, Section 9 goes deep into the Vulnerability Profiler's redesigned logic, and the Awareness Engine's "why/how" explanation generation is woven through Sections 4 to 8, since it draws on exactly the same evidence the Defense Engine uses.

4. The Defense Engine and the Awareness Engine

Score decides the action; the other three only explain it



These never touch the action above — they only feed the explanation below



four-factor model · 4 inputs → 1 severity level → 1 of 4 actions

The core principle behind the Defense Engine is simple to say and important to follow precisely: **the Score decides the action, and nothing else does.** C2 splits its reasoning into two separate jobs so a fast, deterministic action can still be explained well: the **Defense Engine** looks at the risk score alone and maps it to one of four severity levels and actions — nothing more, nothing less. The **Awareness Engine** does the explaining: it draws on everything else C1 and C3 produced to tell the user why that action was taken, how confident the system really is, and what evidence backs it up.

So C2 separates the decision from the explanation. Only one factor feeds the Defense Engine's decision — the risk score itself. The other three factors never influence the action; they exist purely to give the Awareness Engine the material it needs to explain that action clearly:

- **Score** — the risk score handed over by C1 or C3. C1's score is now produced by a trained logistic-regression combiner over its DistilBERT scam probability and its rule-based indicators (malicious URL, impersonation, credential request, authentication result, etc.), calibrated to a score from 0 to 100 — not a hand-picked weighted sum. This is the only factor the Defense Engine uses: it reads the score, places it in a band, and that band is the action. (How C1's and C3's scores differ in exactly what this looks like is covered in Sections 5 and 6.)
- **Evidence Strength** — how many corroborating indicators support the score, and how strong they are individually. This plays no role in the action taken; it feeds the Awareness Engine, which uses it to tell the user how well-supported the verdict is.
- **Reliability** — how complete and trustworthy the detection run itself was. Like Evidence Strength, this never changes the action — it feeds the Awareness Engine, which uses it to tell the user how much confidence to place in the result.
- **AI Authorship Assessment** — whether the email's text shows signs of being AI-generated, taken directly from C1's AI-generation detector (sentence rhythm, word variety, and perplexity, with a confidence score). This factor also never touches the Defense Engine's decision — a Critical-score email is blocked whether or not it was AI-written, and a Low-score email is left alone whether or not it was AI-written. Its only job is to give the Awareness Engine one more piece of explanation: when the text does show AI-authorship signs, the Awareness Engine can say so alongside the score, strengthening the explanation the user sees.

The Defense Engine decides purely from this score, placed into one of four bands: **Low (0–10)** — the email continues to the user normally; **Medium (11–39)** — it continues, but restricted: the user can read it, but cannot click links, download or view attachments, or copy or paste its content; **High (40–69)** — it is quarantined, moved out of the inbox into a safe folder the user rarely checks; **Critical (70–100)** — it is blocked and permanently removed from the inbox, with no way to recover it. The band is the decision — nothing else moves it up or down. Evidence Strength, Reliability, and AI Authorship Assessment never change which band applies; they only shape the explanation the Awareness Engine builds around whichever action the score already decided.

5. How C1 Connects to C2

C1 does not hand C2 a ready-made severity level or a pre-decided action — it hands over the **raw material**, and C2's own four-factor engine is what turns that material into a decision. This matters: C1's job ends at detection, and the reasoning about what to do with that detection belongs entirely to C2.

Why C1's evidence needs this extra step: detecting whether an email's text is an AI-driven scam is inherently a probabilistic judgement — patterns in language, not a direct technical fact. A single score here can be a false positive, so C2 cannot treat it as proof on its own. It needs the evidence behind the score, and a sense of how reliable the detection run was, before deciding how serious the situation really is.

What C1 sends to C2

	Detail
Risk score	A 0–100 value representing how confident C1 is that this is a scam, and specifically an AI-driven one
Corroborating evidence	The specific indicators that justify the score — for example, a failed sender-authentication check, a lookalike domain, a credential-request form, or the stylometric signals (perplexity, burstiness) that pointed to AI authorship
Detection completeness signal	Whether every detection module in C1's pipeline actually ran, or whether something was skipped, degraded, or failed

What C2 does with it

C2 reads this raw material and derives all four factors itself, but routes them to two different places. The **Score** factor comes directly from the risk score and goes straight to the Defense Engine — it is the only input the Defense Engine ever sees. The other three go to the Awareness Engine only: **Evidence Strength** is derived by weighing how many corroborating indicators are present and how strong each one is; **Reliability** is derived from the completeness signal — a score from a fully completed detection run is trusted more than one from a degraded run; and **AI Authorship Assessment** is read directly from

C1's AI-generation detector result (AI-written: yes/no, with a confidence score). None of these four factors arrive pre-computed — C2 is where the reasoning actually happens — but only one of them ever reaches the Defense Engine's decision.

6. How C3 Connects to C2

C3 connects to C2 differently, and deliberately so — it does not go through the four-factor engine at all. Where C1's evidence needs corroboration because text-based scam detection is probabilistic, C3's finding is a direct technical result: its classifier either detects the statistical signature of hidden data in an image, or it doesn't. The confidence score that comes with that verdict is already the thing that matters, so C2 doesn't need to weigh it against separate evidence-strength or reliability factors the way it does for C1.

What C3 sends to C2

	Detail
Prediction	Whether C3 classifies the image as Clean or Stego. Sent to the **Defense Engine** — one of only two C3 outputs that influence the action.
Stego score	The model's confidence score for the stego class. Sent to the **Defense Engine** alongside Prediction — together these two map directly to a severity level.
Clean score	The model's confidence score for the clean class. Sent to the **Awareness Engine** only, to help explain the result — it never affects the action.
Grad-CAM heatmap	A visual explanation of which image regions influenced the prediction. Sent to the **Awareness Engine** only, as visual proof shown to the user — it never affects the action.

That's the whole input from C3 — no separate evidence list, no completeness signal. C3's path is intentionally shorter, and now splits two ways: Prediction and Stego score go to the Defense Engine; Clean score and the Grad-CAM heatmap go to the Awareness Engine.

What C2 does with it

The Defense Engine takes only C3's **Prediction** and **Stego score** and maps them **directly** to one of the same four severity levels (Low, Medium, High, Critical), and from there to the same four defense actions (continue, restrict, quarantine, block) — skipping the four-factor analysis entirely for this path, the same way it skips Evidence Strength, Reliability, and AI Authorship Assessment on C1's path. The **Clean score** and the **Grad-CAM heatmap** never reach the Defense Engine at all — they go straight to the Awareness Engine, which uses them to show the user why the image was flagged, including a visual highlight of the suspicious region.

Why the two paths are different

This is a deliberate design choice, not an inconsistency: C1's evidence is linguistic and probabilistic, so it benefits from corroboration before C2 trusts it enough to act. C3's evidence is a direct technical measurement on the image itself, so the extra corroboration step isn't needed in the same way — the confidence score already carries that weight. Both paths still end at the same place — one of four severity levels, one of four actions — just reached by a different route suited to the kind of evidence involved.

7. Combined Severity: When Both C1 and C3 Fire

A fixed lookup table, not just the higher severity

Rows: C1's severity · Columns: C3's severity · Cell: the combined result

	C3: Low	C3: Medium	C3: High	C3: Critical
C1: Low	Low	Medium	High	Critical
C1: Medium	Medium	Medium	High	Critical
C1: High	High	High	**Critical**	Critical
C1: Critical	Critical	Critical	Critical	Critical

Highlighted cell: two independent High findings escalate one step, to Critical

combined severity rule · higher of the two paths wins, one unified explanation

An email can trigger both paths at once — its text can be flagged as a scam by C1, and its image attachment can be flagged as steganographic by C3, at the same time. When that happens, C2 does not produce two separate verdicts for the user to reconcile themselves. It follows one rule — a fixed lookup table, not a simple comparison:

Combine the two severity levels using a fixed lookup rule — because C1 and C3 are fully independent findings, they are combined deliberately rather than just picking whichever is higher — and present **one unified explanation** that shows both pieces of evidence together, not two disconnected alerts.

In most cases, the rule is to take the higher of the two severity levels. The one exception is when **both paths independently land on High**: two separate findings that each, on their own, say "this is a serious case" are treated as more serious than either alone, so the combined result escalates one step to Critical.

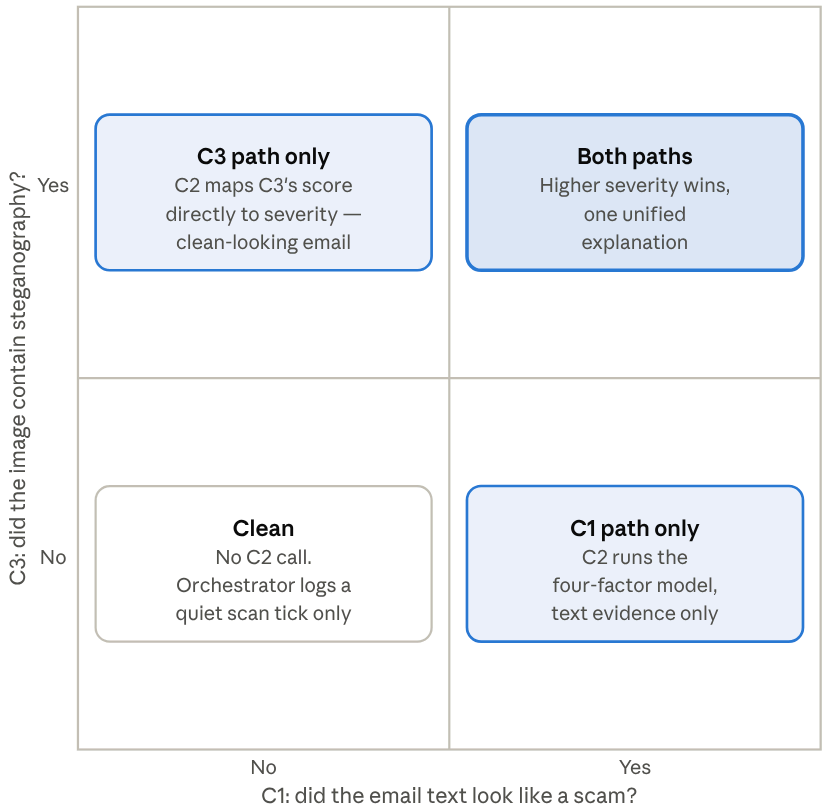
The full rule, as a lookup table (rows = C1's severity, columns = C3's severity):

C1 \ C3	Low	Medium	High	Critical
Low	Low	Medium	High	Critical
Medium	Medium	Medium	High	Critical
High	High	High	Critical	Critical
Critical	Critical	Critical	Critical	Critical

This matrix only applies when both C1 and C3 are active for the same email. When only one of them has produced a result, C2 runs purely off whichever one is active — there's nothing to combine (Section 8 covers this). The user still sees one clear verdict, with the reasoning from both components shown side by side: "This email was flagged for two reasons — the message text shows signs of a credential-phishing attempt, and the attached image was found to contain hidden data." This keeps the "why/how" principle from Section 2 intact even in the combined case: nothing about this unified result is unexplained.

8. Trigger and Invocation Logic

Four outcomes, only one of them stays quiet



trigger logic · 2 independent findings, 4 outcomes, 1 stays quiet

This is the corrected invocation flow, worked through carefully because an earlier version of this logic had a real gap: it gated C3 on C1's scam verdict, which meant a steganographic image on an otherwise clean-looking email would never have been checked at all. The corrected logic treats C1's text analysis and C3's image analysis as genuinely independent checks.

The chain, step by step:

1. **C1 analyzes every email's text, on its own.** It runs its scam-detection pass independently — it no longer checks for attachments or images at all; that job belongs entirely to C3 now.
2. **C3 independently checks every email for image attachments, on its own.** It isn't triggered by C1 or by anything C1 concludes. C3 looks at the email directly, checks whether it carries an attachment, and identifies whether that attachment is an image.

If one is present, C3 starts its process — a perfectly clean-looking email with an image attachment still gets that image checked, and a scam-flagged email with no image never involves C3 at all.

3. **C3 applies a light pre-filter first**: only certain image types, and only above a minimum size, are actually run through the steganography classifier. Anything outside that isn't analyzed.
4. **Each component only forwards a positive finding.** If C1 finds no scam, that result isn't sent to C2. If C3 finds no steganography, that result isn't sent to C2 either. A "nothing found" result from either component stops there — there's nothing for C2 to act on.
5. **C2 only activates when there's at least one positive finding** — from C1, from C3, or both (Section 7 covers the combined case). The only situation where C2 never runs at all is the one where *neither* component found anything.
6. **Even in that fully clean case, the system doesn't go silent.** The Orchestrator still writes a lightweight "scanned, nothing found" record directly to the shared log — without invoking C2's full engine — so the dashboard can show the system is actively working even on a quiet day, without burdening C2 with analysis it has nothing to do.

9. The Vulnerability Profiler, Redesigned

The Vulnerability Profiler turns the user's history of warnings and interactions into a running picture of whether they're actually getting safer over time — computed from a transparent, documented formula, never a trained model or a hidden score. That principle stays exactly as it is. What's redesigned here is the formula itself, to fix four real weaknesses found in the original version.

How it currently works

The profiler splits the user's interaction history into an "earlier half" and a "recent half," compares the safe-action rate between the two, and assigns one of seven plain-English states: **Insufficient Data**, **Early Observation**, **Improving**, **Consistently Safer**, **Needs Support**, **Declining**, or **Stable/Mixed** — each with a one-line explanation of exactly how it was computed.

The four weaknesses found

Weakness	The problem
Fragile at small sample sizes	A trend verdict can be given off as few as 2 interactions per half — one different click can flip "Improving" to "Declining" on noise, not a real pattern
Blind to *which* scam type the user struggles with	Tracks category counts, but the evolution state itself is one global number — someone safe against phishing but vulnerable to romance scams gets the same label as someone randomly inconsistent
No severity weighting	Ignoring a Critical warning counts exactly the same as ignoring a Low one
A brittle "Needs Support" trigger	Only fires when the recent safe-action rate is *exactly* zero — a user who's mostly, but not entirely, struggling doesn't trigger it

The fixes

- **A minimum sample size per half, not just in total.** Require at least 3 interactions in *each* half before giving a trend verdict — below that, stay in Early Observation with an honest "not enough recent data yet" message, rather than a confident-sounding label built on noise.
- **A per-category safe-action rate, shown alongside the global one.** This is the single most valuable fix: instead of just "Stable/Mixed," the profile can say *"You're consistently safe against phishing (92%), but have acted on 3 of the last 4 romance-scam warnings."* This turns the profiler from a general mood-ring into something the Awareness Engine can use directly — it now knows exactly what to reinforce next.
- **Severity-weighted risky actions.** Ignoring or acting on a warning is weighted by the severity of the event it responded to (a small fixed weight per band — Low, Medium, High, Critical) — so a Critical miss counts for more than a Low one. Still a fixed, documented formula, not a trained model — the "no hidden weights" promise is kept.
- **A rate-based "Needs Support" trigger**, not an exact-zero check — for example, a recent safe-action rate at or below 20% with at least 2 risky or ignored events, so it catches "mostly struggling," not just the perfect-zero edge case.

- **A confidence/sample-size line shown with every state**, always — "Improving (based on 9 recent interactions)" — so the user can see exactly how much to trust the label, the same transparency discipline the rest of C2 is built on.

10. Rule-Based vs. Machine Learning — the Decision

Not every part of C2 should be built the same way, and this was deliberated carefully rather than defaulting to "use ML everywhere" or "use no ML at all."

Engine	Decision	Why
Defense Engine	Fully rule-based. No training.	This decision has real consequences — blocking a legitimate email is a costly mistake. A security action needs to be deterministic and explainable on demand. An ML model here would quietly undermine the "why/how" transparency the whole of C2 is built around — you can't give a clean answer for "why was this blocked" if the decision came from a model's internal weights rather than a named rule.
Vulnerability Profiler	Fully rule-based. No training.	Same transparency reasoning, plus a privacy reason: this component is explicitly designed to never build a hidden behavioral profile of the user. A trained model predicting "user risk" from behavior edges toward exactly that. The documented formula is the feature, not a limitation.

Engine	Decision	Why
Awareness Engine	Lightweight, gated machine learning, used narrowly	Classifying which of 9 scam categories a piece of evidence resembles, and which psychological trigger is present, is a pattern-matching task that benefits from learning across many examples in a way a hand-written rule list struggles to generalise for. This is the one place ML genuinely earns its place.

How the Awareness Engine's ML is scoped

The classifier is deliberately lightweight — a TF-IDF-plus-linear-classifier model, not a heavy transformer — consistent with the same "lightweight, not a second heavyweight model" philosophy already used in C1's own AI-generation detector. It is used for one narrow job only: **picking which pre-written explanation template to show**, never generating free text. And it is **gated**: the model's output has to pass an allow-list and evidence-consistency check before it's trusted; if it fails that check, the system falls back to the deterministic, evidence-priority template selection instead of guessing.

Datasets needed

Only the Awareness Engine's two classifiers need training data — the Defense Engine and Vulnerability Profiler need none, since both are formula-driven:

- **A labelled scam-category dataset** — examples tagged with one of the 9 categories (phishing, romance scam, investment fraud, authority impersonation, tech-support scam, lottery/prize scam, business email compromise, extortion, impersonation).
- **A labelled psychological-trigger dataset** — the same or overlapping examples, tagged with the trigger present (urgency, authority, scarcity, reciprocity, social proof, fear/threat, greed, curiosity).

Two realistic sources, best combined: **public phishing/scam corpora** (Kaggle phishing-email datasets, academic social-engineering datasets) as a starting base — though these skew heavily toward generic phishing and under-represent categories like romance scams, so category balance needs active attention — and, over time, **the system's own detection history**, hand-labelled with category/trigger after the fact and pseudonymised

the same way all other stored data already is. The system's own history is the better long-term source because it reflects real-world cases the detector actually encounters, but it only builds up volume once the system has been running for a while.

11. The Dashboard (Frontend) Design

The dashboard is where everything C2 produces actually reaches the user, so it follows the same guiding rule as the rest of this document: **never show a bare number or label without the reasoning behind it being one click away.** It's built from six panels, ordered from what a user needs at a glance down to what they'd only check occasionally.

#	Panel	What it shows	Where the data comes from
1	**Status hero**	One clear state at the top — "Protected," or "3 threats handled this week" — a single glance answer to "is this working and am I safe," updated live	The live event stream
2	**Recent activity feed**	A reverse-chronological list of flagged events only — time, a one-line plain summary, a colour-coded severity badge, and the action taken	The event/audit log
3	**Event detail** (click into any feed row)	The full "why/how" — category, the specific evidence/indicators, the four-factor reasoning behind the severity, and for a stego finding, the image with its heatmap overlaid and clearly labelled provisional; includes a "this wasn't right" / mark-as-reported control	The full event record

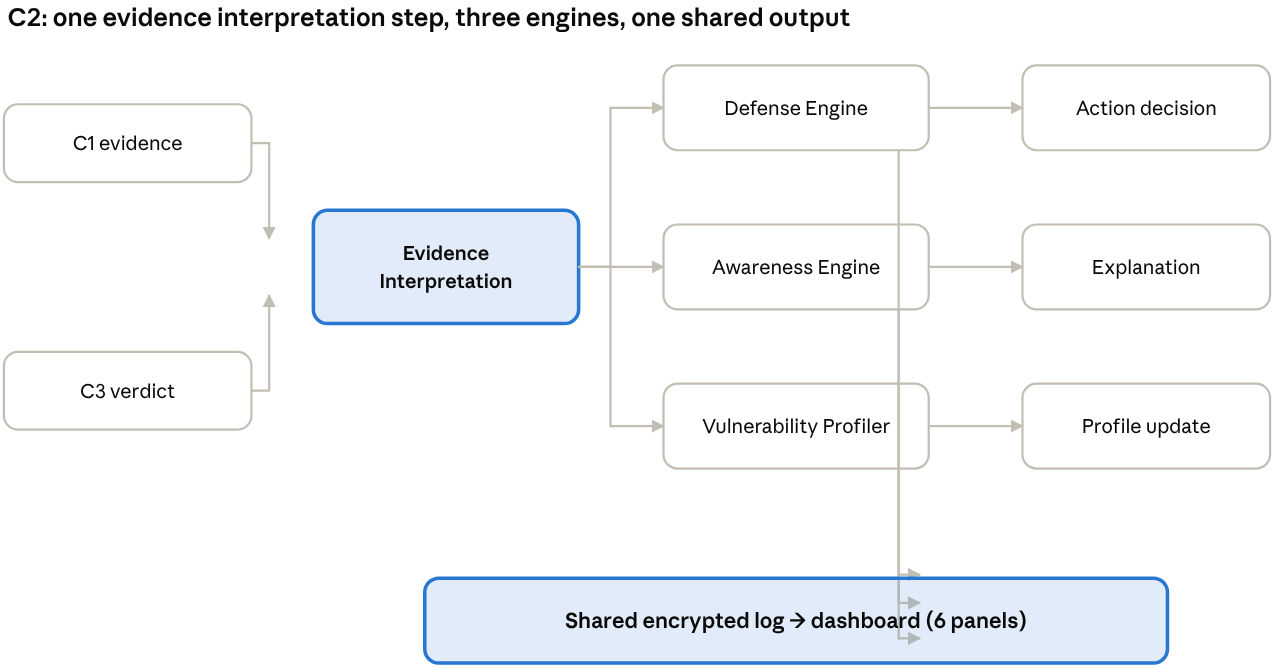
#	Panel	What it shows	Where the data comes from
4	**Your Security Profile**	The Vulnerability Profiler's output, visualised: the current evolution state with its one-line formula explanation, a simple trend line over time, and — with the redesign from Section 9 — a per-category breakdown showing which scam types the user handles well versus where they're weakest	The profile endpoint
5	**Learn / awareness reference**	Short, plain reference cards per scam category, browsable proactively rather than only reactive to alerts, with the weakest category from the profile gently surfaced as a prompt	The awareness template catalogue
6	**System status strip**	Whether C1/C2/C3 are healthy, last scan time, and total scanned vs. flagged — secondary, reassurance-level information, not something a user checks daily	Health and metrics endpoints

Two design choices worth calling out specifically:

Why the status hero comes first. A user who opens the dashboard wants one question answered immediately — "am I okay" — before anything else. Burying that under a table of raw numbers would work against the entire "user-friendly, never confusing" goal this whole component exists to serve.

Why the feed only shows flagged events, not every scanned email. With the corrected trigger logic from Section 8, a fully clean email produces only a lightweight scan tick, not a full C2 record. Showing every one of those on the main feed would bury the events that actually matter under noise — the scan-tick count instead feeds the quieter system status strip at the bottom, where it belongs.

12. Full C2 Architecture



C2 full internal architecture · 2 inputs, 1 interpretation step, 3 engines, 1 shared output

This is the complete internal picture of C2, bringing together everything covered in this document into one view:

- **Two inputs** arrive, each along the path described in Sections 5 and 6 — C1's raw evidence, or C3's direct verdict, or both at once (Section 7).
- **One evidence-interpretation step** receives whatever arrived and prepares it — this is where the "both fire, take the higher severity" rule and the positive-findings-only filtering from Section 8 are actually applied, before anything reaches the three engines.
- **Three engines run from that same interpreted evidence**: the Defense Engine (Section 4) decides the action, the Awareness Engine (Sections 2 and 10) generates the explanation, and the Vulnerability Profiler (Section 9) updates the user's security profile. They run independently of each other — none waits on another — but all three draw from the same evidence, which is what keeps the action, the explanation, and the profile update always consistent with one another.
- **Three outputs converge on one shared, encrypted log**, which is what feeds every panel of the dashboard described in Section 11.

Nothing in this diagram is a separate, disconnected process — it's one pass, through one component, from one piece of evidence to a fully explained, logged, and displayed result.

13. Summary and Design Principles

Everything in this document traces back to a small number of principles, repeated here together because they're what should guide any future change to C2:

1. **Score decides the action; everything else explains it.** The Defense Engine acts on the risk score alone — Evidence Strength, Reliability, and AI Authorship Assessment never change the action, only the explanation the Awareness Engine builds around it. This keeps the action fast and predictable, which matters even more now that a Critical verdict permanently deletes the email, with no undo — the explanation layer is what lets the user understand, and react appropriately, before anything reversible (Medium, High) is lost.
2. **Why, how, and logical — always together.** Every verdict C2 produces comes with the evidence behind it and the reasoning that connects that evidence to the conclusion. A label with no reasoning attached is exactly the failure mode C2 exists to fix.
3. **C1 and C3 are evidence sources, not decision-makers.** Neither hands C2 a ready-made verdict. C1 sends raw evidence for C2's four-factor engine to reason over; C3 sends a direct, already-reliable technical result that maps straight to severity. Two different paths, because the two kinds of evidence are genuinely different in nature — not an inconsistency.
4. **Nothing stays silent, but nothing is wasted either.** C2 only runs its full reasoning when there's something to reason about; a clean email still leaves a quiet trace so the user can trust the system is actually working.
5. **Rule-based where it matters, ML where it earns its place.** Decisions with real consequences (the action, the profile) stay deterministic and auditable. Machine learning is used only where it's structurally the right tool — category and trigger classification — and even there, it's lightweight and gated with a deterministic fallback.
6. **The dashboard is the user's only window into all of this**, so it inherits the same rule as everything above it: nothing shown without the reasoning one click away.

Taken together, these six principles are what make C2 something genuinely different from a conventional silent-blocking security tool — not because it detects more, but because every decision it makes is one the user can actually understand, question, and learn from.

14. The Story: How Component 2 Handles One Email

Everything above explains the pieces on their own. This section tells the same system as a story — one email, moving through C2, step by step, in plain language.

The email arrives

Picture an inbox. A new email lands. Thunderbird hands it to the system.

Two things happen at the same time, completely separately:

C1 reads the words in the email. It is checking: does this sound like a scam?

C3 checks whether the email has an image attached. If it does, C3 looks closely at that image. It is checking: is something hidden inside this picture?

C1 does not wait for C3. C3 does not wait for C1. They work independently, and each one only speaks up if it actually finds something.

Case 1 — A normal, safe email

Say the email is just a real message from a colleague. No scam language, no hidden image.

C1 reads it, finds nothing alarming, and stays quiet. C3 either finds no image, or finds an image with nothing hidden in it, and also stays quiet.

Because nothing was flagged, C2 has nothing to act on. The email goes straight to the inbox. A small, quiet record is still kept, so later the user can see the system did check this email and found it safe — but the user is never bothered with it.

Case 2 — A scam email, text only

Now say the email is a phishing attempt — it asks the user to click a link and enter their password.

C1 reads the text and becomes suspicious. It produces four things:

A **Score** from 0 to 100 — how confident C1 is that this is a scam. Say it comes out to 55.

Supporting **Evidence** — for example, the link goes to a lookalike domain, and the email asks for a password.

Its own **Reliability** — did every part of C1's analysis actually finish properly, or did something get skipped? Say everything ran fine, so reliability is high.

Whether the writing itself looks AI-generated, from its own separate AI-detector — an **AI Authorship** signal. Say the writing does show signs of being AI-written.

All four of these go to C2. Here is the important part: C2's Defense Engine only looks at one of them — the Score. It sees 55, which falls in the High band (40–69), so its decision is immediate and simple: quarantine the email — move it out of the inbox into a safe folder the user rarely opens.

The other three — Evidence, Reliability, AI Authorship — never touched that decision. They go somewhere else: the Awareness Engine, the part of C2 that explains things to the user. It takes those three and writes something the user can actually understand: "This email was quarantined because it scored 55 out of 100 for scam risk. It links to a lookalike domain and asks for your password. The detection ran completely, so this result can be trusted, and the writing style matches AI-generated text — all of which point the same way."

So: one number decided the action. Three other things explained why.

Case 3 — A hidden image, text looks fine

Now imagine a different email. The written message looks completely ordinary — nothing C1 flags. But it has an image attached, and hidden inside that image is secret data.

C1 stays quiet — nothing in the text looks wrong.

C3 examines the image and produces four things: a **Prediction** (Clean or Stego — here, Stego: something is hidden), a **Stego score** (how confident C3 is in that call — say 80), a **Clean score** (its confidence in the opposite answer), and a **Grad-CAM heatmap** (a picture highlighting exactly which part of the image looks suspicious).

Just like before, C2's Defense Engine only uses two of these — Prediction and Stego score — and maps them straight to a severity level. A Stego verdict at 80% confidence lands in the Critical band, so the email is blocked and permanently removed — no undo.

The Clean score and the heatmap never reach the Defense Engine. They go to the Awareness Engine instead, which uses the heatmap to show the user exactly which part of the image triggered the alert — real, visual proof, not just a claim.

Case 4 — Both fire at once

Now the hardest case: the text looks somewhat suspicious to C1, and the image also looks suspicious to C3 — at the same time, completely independently of each other.

Say C1's text analysis comes back Medium. Say C3's image analysis comes back High.

C2 does not just pick the bigger of the two and move on. It looks both severities up in a fixed table. Medium and High together land on High. The email is quarantined, and the user sees one explanation covering both reasons — the wording in the message, and the finding in the image.

There is one special case in that table worth telling as its own small story: if C1 says High and C3 also, independently, says High, C2 does not just call it High. Two separate systems, looking at two completely separate things — the words and the image — both independently raising the same serious alarm is treated as more dangerous than either one alone. So that one combination escalates a step further, to Critical, and the email is blocked for good.

How the Awareness Engine picks what to say

Once the Defense Engine has already picked the action, the Awareness Engine still has one more job: choosing exactly which explanation to show. It keeps a set of pre-written, plain-English explanation templates — one for each kind of flagged email. A small, lightweight classifier helps pick the right template, but it is never allowed to write its own sentences, and it is never trusted blindly: whatever it picks has to pass a check against the actual evidence first. If that check fails, the system falls back to a simple, deterministic rule instead of guessing. This way, nothing the user reads is ever made up — it is always one of a known, checked set of explanations.

After the action: the Dashboard

Everything that happened — the action taken, the explanation, the evidence — ends up in one place the user can actually look at: the dashboard. The very first thing it shows, before any numbers or details, is one simple answer to the question "am I okay right now?" Underneath that sits a feed of only the emails that were actually flagged — a totally clean email doesn't clutter this feed, it just quietly adds to a small "scanned" counter at the bottom. Nothing on the dashboard is ever just a bare number — every score, every label, has its reasoning one click away, the same rule that runs through everything else in this system.

Keeping score over time: the Vulnerability Profiler

There's one more part of the story that isn't about any single email — it's about the user, over time. Every time the user reacts to a warning — ignoring it, heeding it, clicking through anyway — that gets quietly recorded. The Vulnerability Profiler looks at this history and works out, using a plain, documented formula — never a hidden model — whether the user is actually getting safer or not. It splits their history into an earlier half and a more recent half, compares how safely they acted in each, and settles on one of seven honest, plain-English labels — things like "Improving," "Needs Support," or "Declining" — each one coming with a one-line explanation of exactly how it was worked out. This is what turns a pile of individual email decisions into one running picture of how well the user is actually doing.

What the user actually sees

Whatever happened above, the user never sees a raw number or a wall of technical detail. They see one plain decision — continue, continue but restricted, quarantined, or blocked — and one plain explanation next to it, built by the Awareness Engine, showing exactly why. If a Critical-level email was permanently removed, that is made very clear, because there is no undoing it.

Those two things — the dashboard view in the moment, and the Vulnerability Profiler's view over time — are what the user actually experiences, on top of the one-email story told above. Sections 9 and 11 cover the profiler and the dashboard in full.